

Modern C++20 Custom Polymorphic Memory Resources

High-Frequency Trading and Low-Latency Systems Architecture

1. Memory Allocation Overhead in Real-Time Systems

Standard heap allocations via malloc/new introduce variable latency due to kernel calls and heap fragmentation.

2. Custom Arena Pool Allocator Implementation

```
template <typename T, size_t BlockSize = 4096>
```

```
class FixedArenaAllocator {
```

```
public:
```

```
    using value_type = T;
```

```
    T* allocate(size_t n) {
```

```
        if (offset_ + n * sizeof(T) > BlockSize) throw std::bad_alloc();
```

```
        T* ptr = reinterpret_cast<T*>(&buffer_[offset_]);
```

```
        offset_ += n * sizeof(T);
```

```
        return ptr;
```

```
    }
```

```
    void deallocate(T* p, size_t n) noexcept { /* no-op in arena */ }
```

```
private:
```

```
    alignas(std::max_align_t) std::byte buffer_[BlockSize];
```

```
    size_t offset_{0};
```

```
};
```

3. Performance Benchmark Summary

Arena allocation achieves O(1) allocation speed with zero heap locks.